

Book Recommender

Manuale tecnico

Sommario

1. Introduzione

1.1 Tecnologie utilizzate.....	4
1.2 Obiettivi del sistema.....	4

2. Progettazione della soluzione

2.1 Architettura del sistema.....	4
2.2 Componenti principali.....	5

3. Documentazione UML del Sistema

3.1 Class Diagram.....	6
3.2 Sequence Diagram.....	7-12
3.3 State Diagram.....	13

4. Progettazione del DATABASE

4.1 Processo di progettazione.....	14
4.2 Schema relazionale implementato.....	16
4.3 Tabelle principali.....	16-21

5. Architettura e Implementazione

5.1 Scelte architetturali.....	22
5.2 Gestione della comunicazione client-server.....	25
5.3 Operazioni Log In e Register.....	27
5.4 metodi di ricerca.....	29

6. Limiti della soluzione sviluppata

6.1 Limiti di sicurezza.....	30
6.2 Limiti funzionali.....	30
6.3 Limiti interfaccia grafica.....	30
6.4 Limiti del sistema operativo.....	30

7. Sitografia

7.1 tecnologie utilizzate.....	31
7.2 programmazione socket e multi-thread.....	31
7.3 DATABASE e UML.....	31
7.4 GIT e GitHub.....	31
7.5 sviluppo GUI.....	32

1 Introduzione

Book Recommender è un'applicazione sviluppata in Java che implementa un sistema di gestione e raccomandazione di libri. Il sistema è stato riprogettato per utilizzare un'architettura client-server con persistenza dei dati su database PostgreSQL, sostituendo il precedente approccio basato su file locali.

Il progetto è sviluppato in Java 21, usa un'interfaccia grafica costruita con Java Swing ed è stato sviluppato e testato su sistema operativo Windows 10.

1.1 Tecnologie utilizzate:

Per lo sviluppo di questo progetto sono state utilizzate diverse tecnologie tra cui:

- Java 21+: Linguaggio di programmazione principale
- Java Swing: Framework per l'interfaccia grafica utente
- PostgreSQL: Sistema di gestione database relazionale
- JDBC: API per la connessione al database
- Supabase: Servizio cloud per hosting del database
- Socket Programming: Comunicazione client-server

1.2 Obiettivi del sistema:

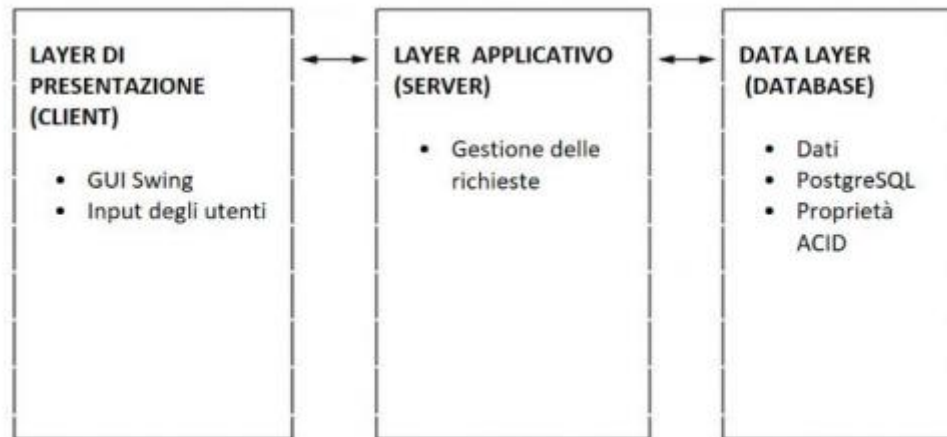
- Fornire un sistema scalabile di gestione e raccomandazione libri
- Implementare un'architettura robusta client-server
- Garantire persistenza e integrità dei dati
- Supportare accesso concorrente di più utenti

2. progettazione della soluzione

2.1 architettura del sistema:

Il sistema implementa un'architettura client-server a tre livelli:

- Presentation Layer (Client): Interfaccia grafica Swing
- Application Layer (Server): Logica e gestione richieste
- Data Layer (Database): Persistenza dati PostgreSQL



2.2 Componenti principali:

Componenti Server-Side:

- Server.java: Server principale che gestisce le connessioni
- ServerMulti.java: Thread worker per gestire singole connessioni client

Componenti Client-Side:

- Classi GUI: Interfacce grafiche per interazione utente:
 - BookINFO
 - CsvTable
 - PersonalLibrary
 - SearchPanel
 - UserPanel
 - ToolBar
 - LogInPanel
 - Register
- Proxy: Gestione comunicazione con server

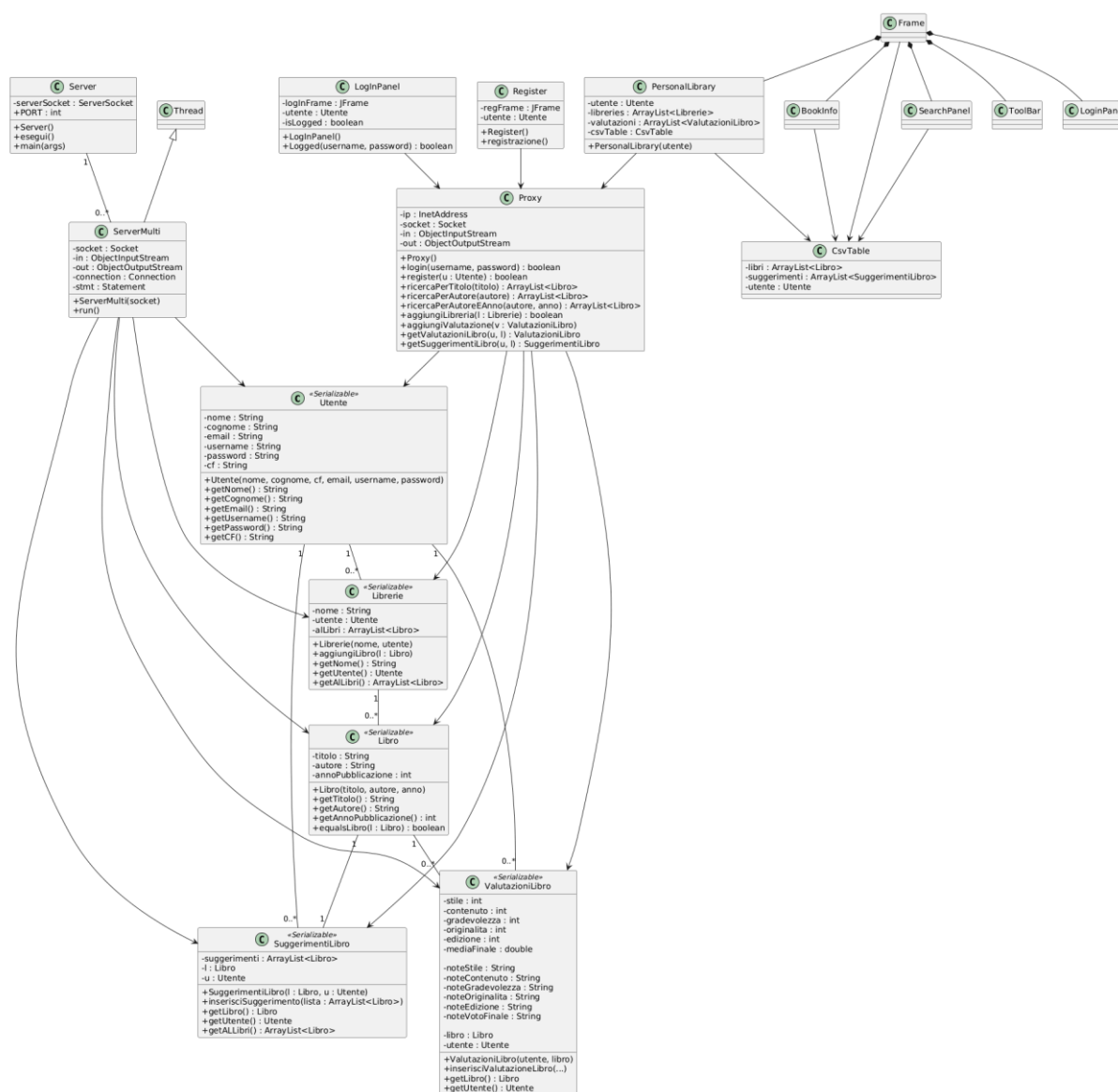
Utente, Libro, Librerie, SuggerimentiLibro e ValutazioniLibro

3. Documentazione UML del Sistema

La documentazione UML fornisce una rappresentazione formale dell'architettura software a diversi livelli di astrazione. I diagrammi UML sono stati sviluppati per catturare sia gli aspetti statici che dinamici del sistema, fornendo una visione completa dell'architettura client-server implementata.

3.1 Class Diagram

Il Class Diagram mostra l'architettura delle classi del dominio della Book Recommender App, evidenziando le relazioni tra le entità principali (Utente, Libro, ValutazioniLibro, SuggerimentiLibro, Proxy, ServerMulti, Server, Thread, LoginPanel, Register, PersonalLibrary, BookInfo, SearchPanel, Toolbar, LoginPanel, CsvTable).

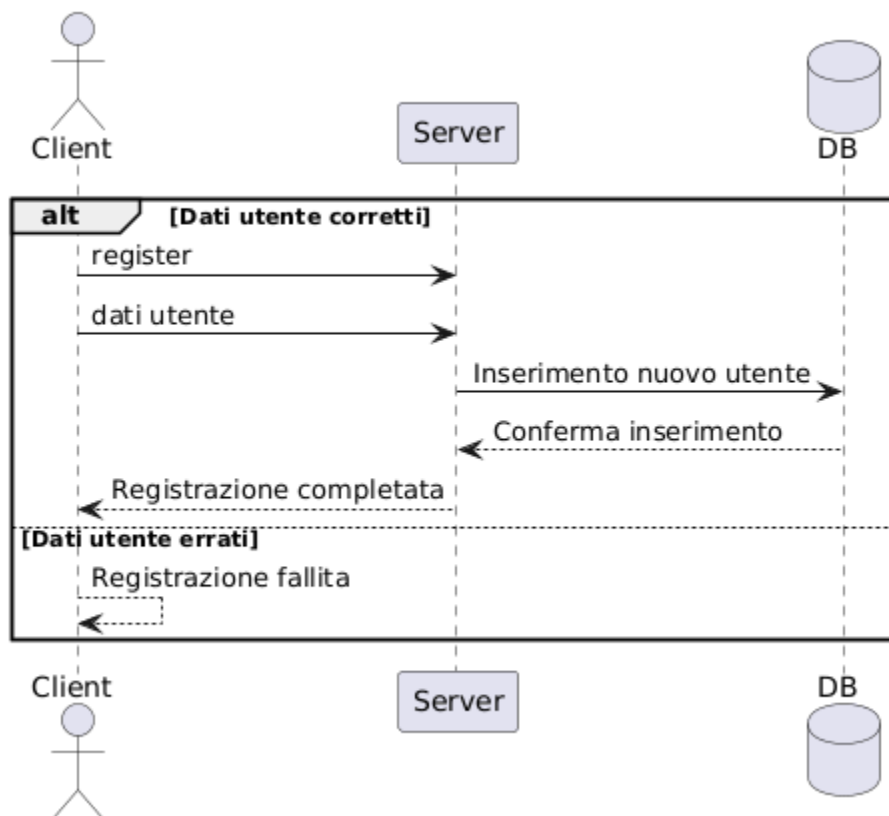


Autore, Libreria, Valutazione, Consigli) e definendo gli attributi e metodi necessari per la gestione delle funzionalità di raccomandazione e valutazione dei libri:

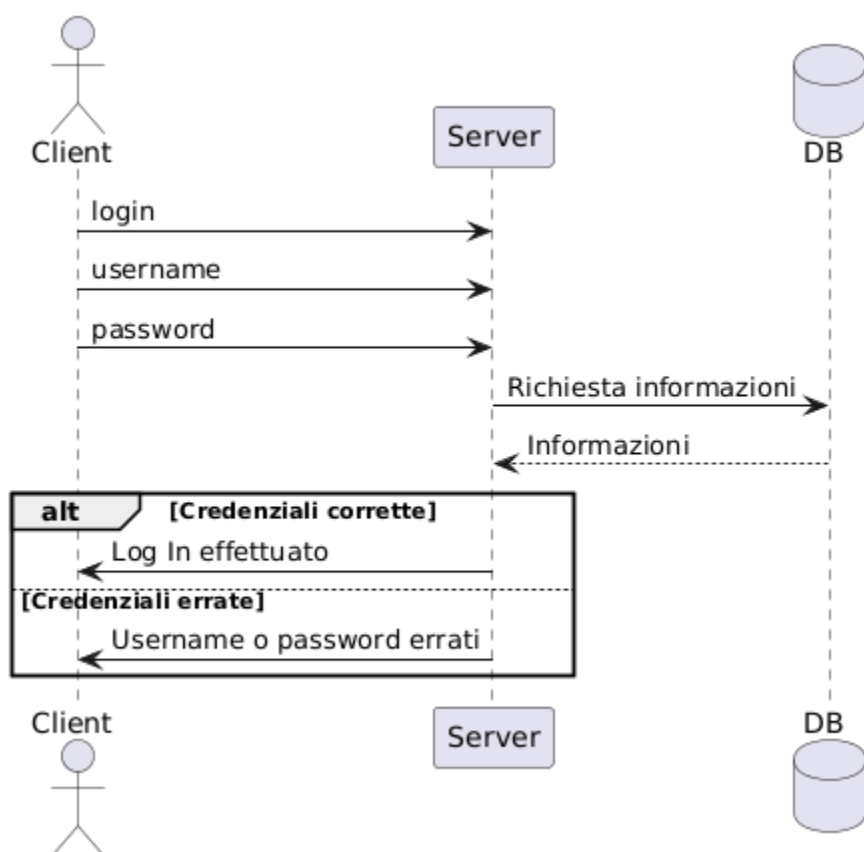
3.2 Sequence Diagram

Il Sequence Diagram rappresenta il flusso di comunicazione temporale tra il client e il server nelle operazioni fondamentali dell'applicazione, come l'autenticazione utente, la ricerca di libri nel database PostgreSQL e la sincronizzazione delle librerie personali attraverso l'architettura client-server:

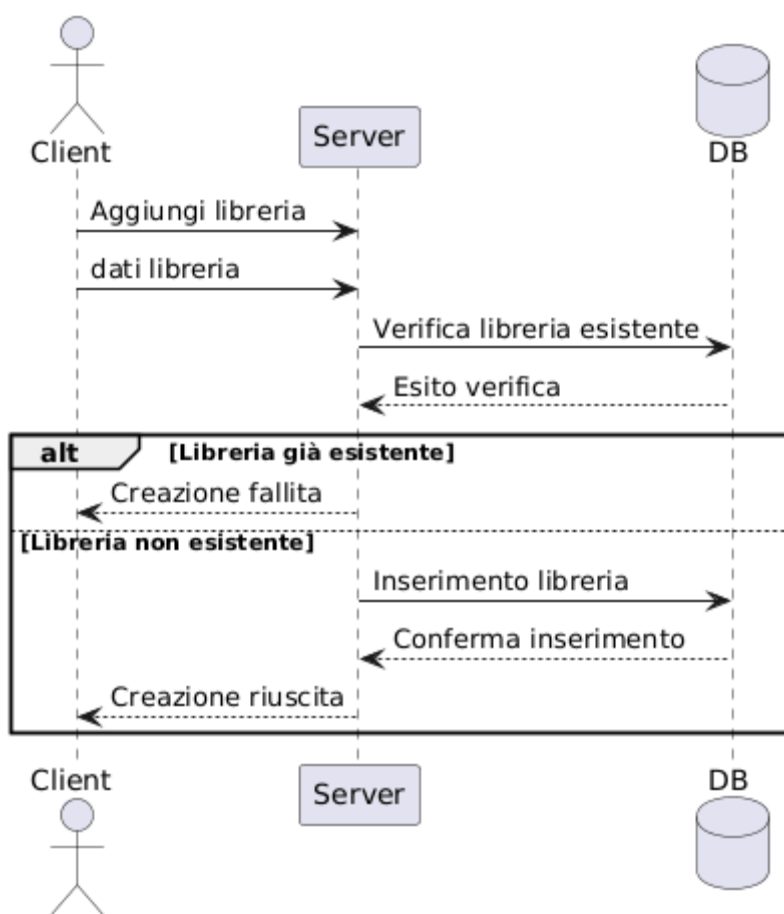
Register



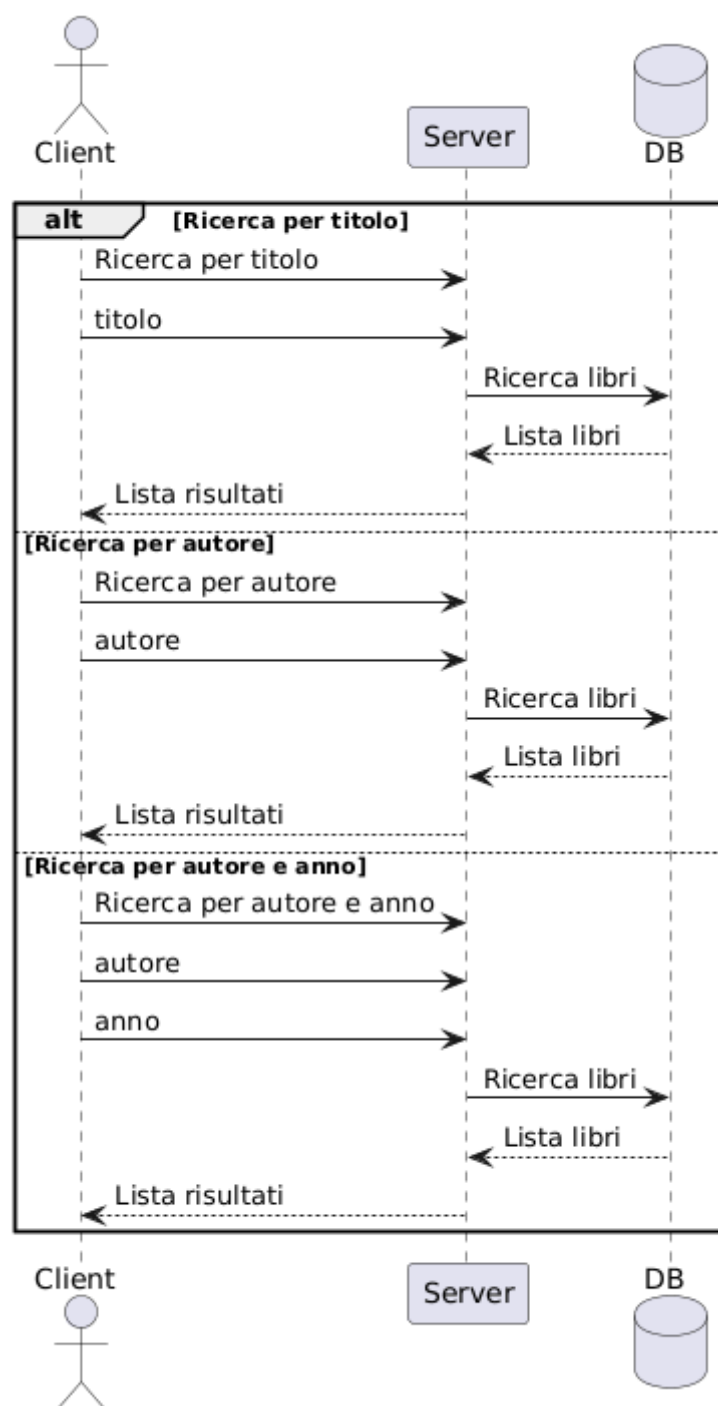
Log In



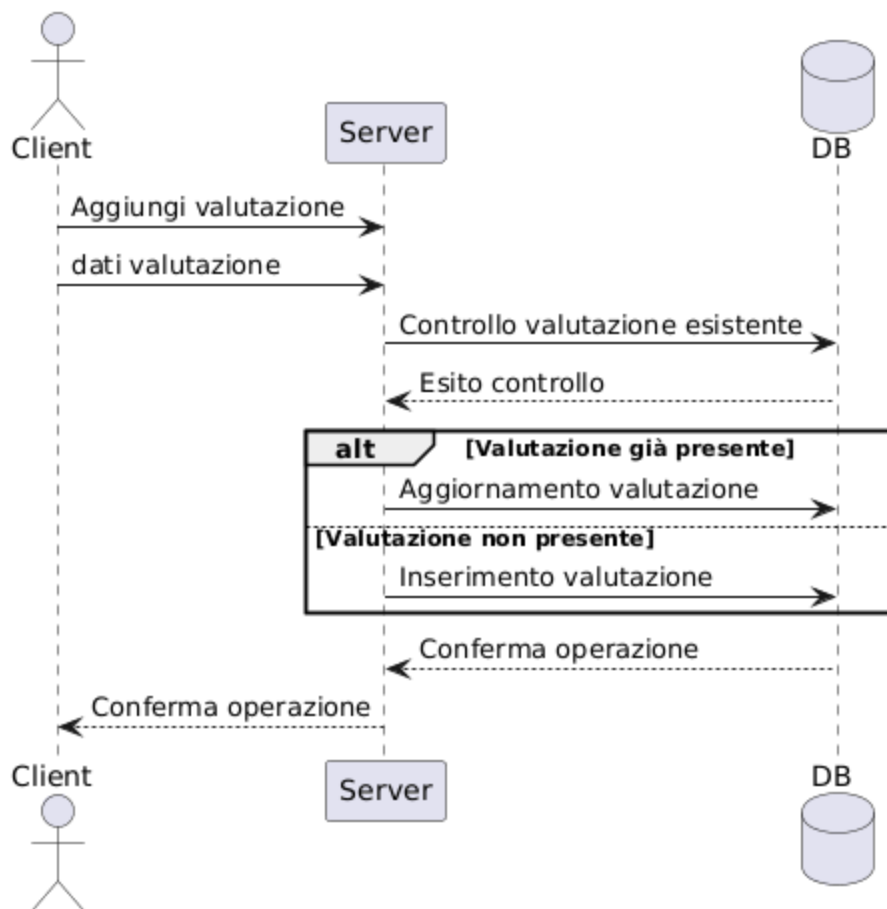
Aggiungi Libreria



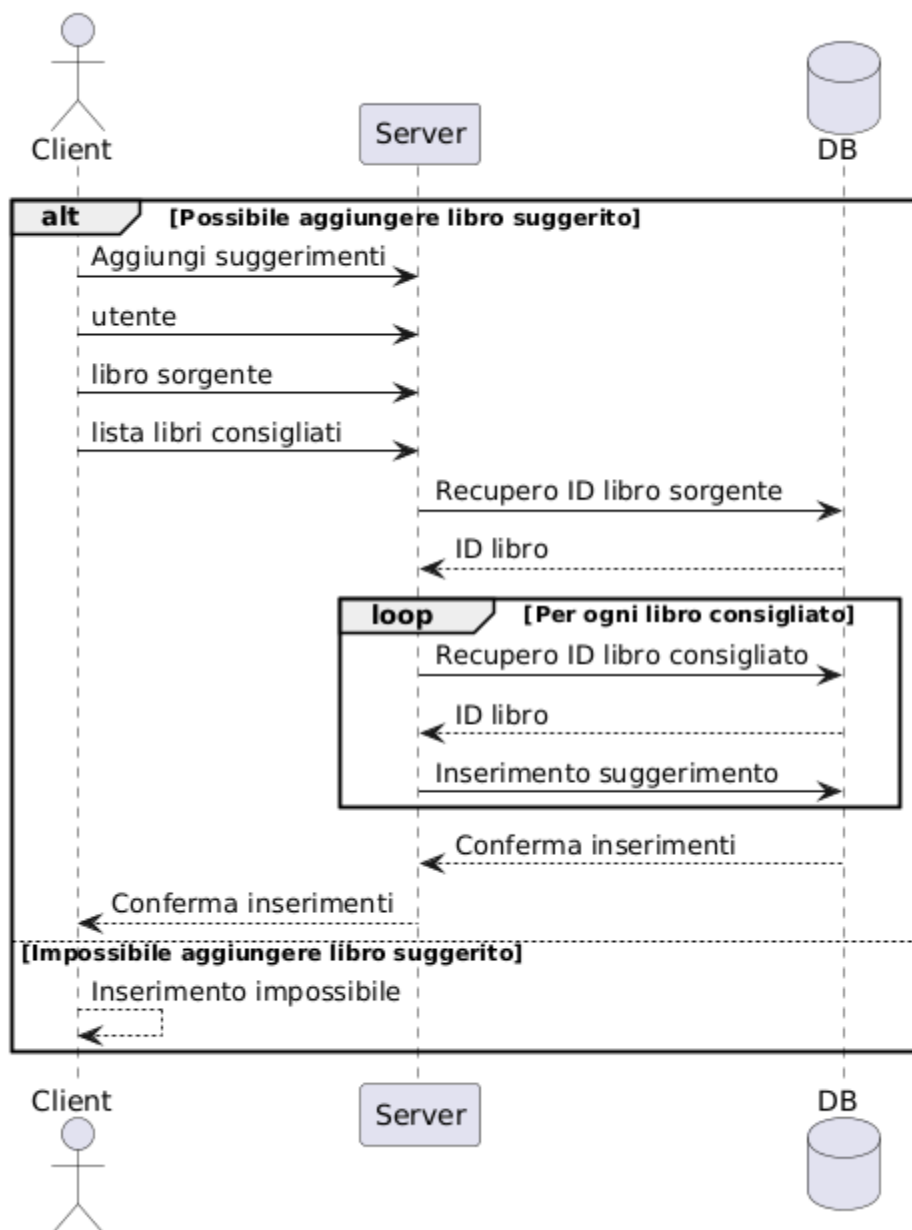
Ricerca per titolo, per autore e per autore e anno



Aggiungi valutazione

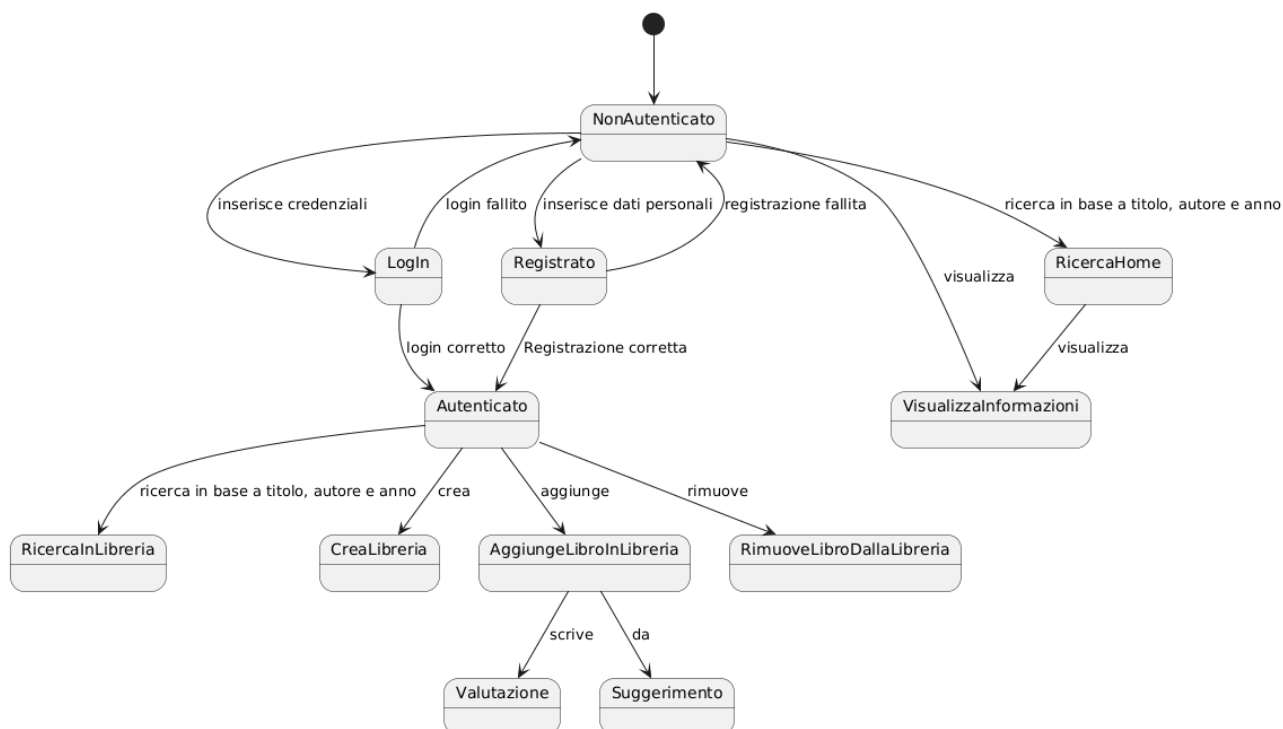


Aggiungi suggerimenti



3.3 State Diagram

Lo State Diagram modella i diversi stati di una sessione utente nel Book Recommender, dalla connessione iniziale al server fino alla gestione delle operazioni di ricerca e valutazione, evidenziando le transizioni di stato in base alle azioni dell'utente.



4 La progettazione del DATA BASE

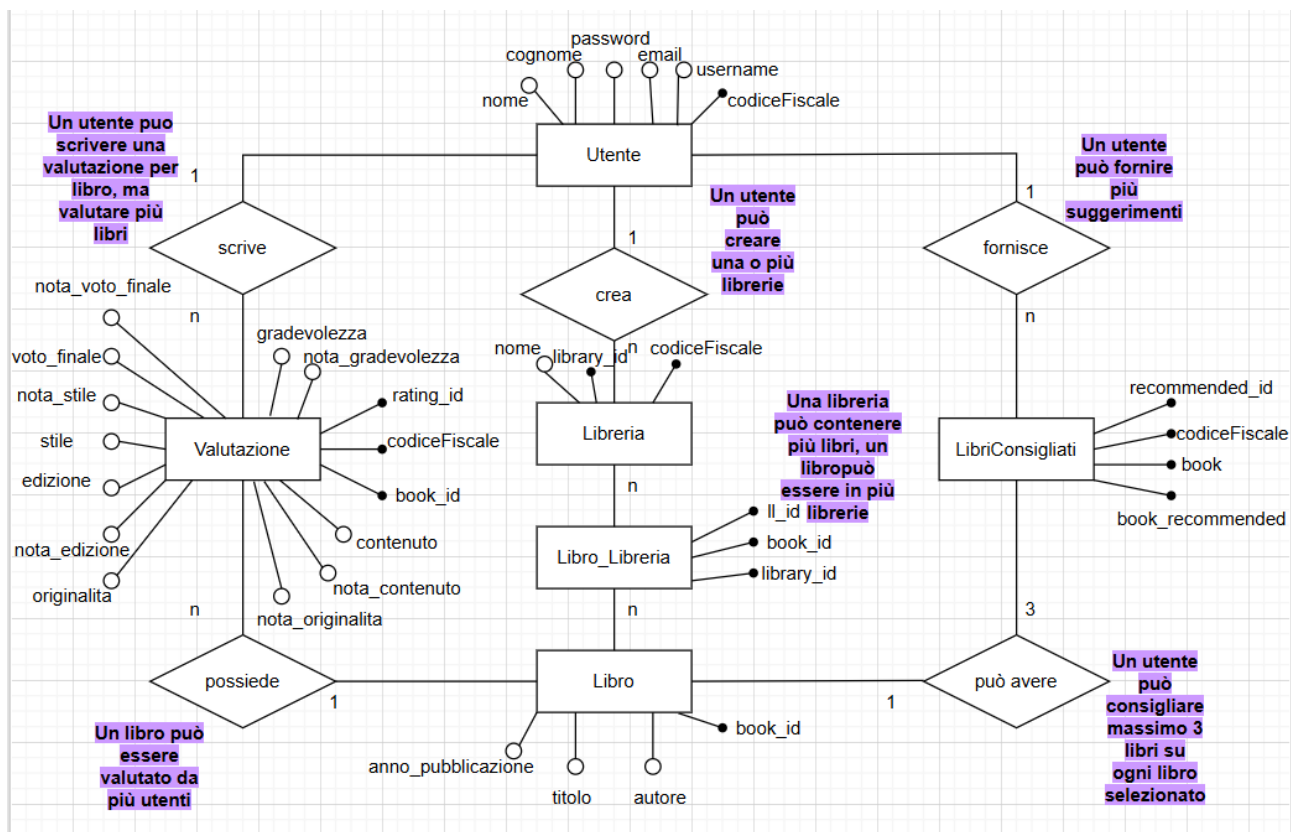
4.1 processo di progettazione

La progettazione del database ha seguito un approccio metodico strutturato in diverse fasi:

- 1. Analisi dei Requisiti:** Il processo è iniziato con l'identificazione delle entità principali del dominio applicativo e delle loro relazioni. Sono stati analizzati i requisiti funzionali per determinare le operazioni che il sistema deve supportare.
- 2. Schema Entità-Relazioni (ER) È stato sviluppato uno schema ER che modella:**
 - a. Le entità principali del dominio
 - b. Le relazioni tra entità con cardinalità appropriata
 - c. Gli attributi di ogni entità con domini e vincoli
 - d. Le chiavi primarie
- 3. Modellazione Comportamentale:** In parallelo alla modellazione ER, sono stati sviluppati diversi diagrammi UML.
- 4. Processo di Trasformazione ER → Relazionale:**

La trasformazione dallo schema ER al modello relazionale ha comportato diverse decisioni progettuali:

 - a. Mappatura Entità:** Ogni entità del modello ER è stata trasformata in una tabella relazionale
 - b. Gestione Relazioni:** Le relazioni many-to-many sono state risolte attraverso tabelle di congiunzione
 - c. Chiavi Primarie:** Scelta tra chiavi naturali (es. codice fiscale) e chiavi surrogate (es. ID sequenziali)
 - d. Vincoli di Integrità:** Implementazione di tutti i vincoli identificati nel modello ER



4.2 Schema relazionale

implementato Il database è stato implementato su **PostgreSQL** utilizzando la piattaforma cloud Supabase, che offre:

- Hosting gestito del database PostgreSQL
- Interfaccia web per gestione e monitoraggio
- Backup automatici e alta disponibilità

4.3 Tabelle principali

Utente

```
1 CREATE TABLE Utente (  
2     nome VARCHAR(50) NOT NULL,  
3     cognome VARCHAR(50) NOT NULL,  
4     email VARCHAR(100) NOT NULL UNIQUE,  
5     codiceFiscale CHAR(16) PRIMARY KEY,  
6     password VARCHAR(255) NOT NULL,  
7     username VARCHAR(50) NOT NULL  
8 );
```

Descrizione:

Anagrafica completa degli utenti registrati al sistema.

Funzione:

Memorizza le informazioni personali e di autenticazione degli utenti che utilizzano l'applicazione.

Chiave primaria:

codiceFiscale

Dipendenze:

È collegata a:

- **Libreria** tramite codiceFiscale
- **Valutazione** tramite codiceFiscale
- **LibriConsigliati** tramite codiceFiscale

Libro

```
1 CREATE TABLE Libro (  
2     book_id SERIAL PRIMARY KEY,  
3     titolo VARCHAR(200) NOT NULL,  
4     autore VARCHAR(100) NOT NULL,  
5     anno_pubblicazione INT  
6 );
```

Descrizione:

Catalogo completo di tutti i libri presenti nel sistema.

Funzione:

Memorizza le informazioni bibliografiche di ogni libro (titolo, autore, anno di pubblicazione).

Chiave primaria:

book_id (autoincrementale)

Dipendenze:

È collegata a:

- **Libro_libreria** tramite book_id
- **Valutazione** tramite book_id
- **LibriConsigliati** tramite book_id e book_recommended

Libreria

```
1 CREATE TABLE Libreria (  
2     library_id SERIAL PRIMARY KEY,  
3     nome VARCHAR(100) NOT NULL,  
4     codiceFiscale VARCHAR(16) NOT NULL,  
5     FOREIGN KEY (codiceFiscale) REFERENCES Utente(codiceFiscale)  
6 );
```

Descrizione:

Rappresenta le librerie personali create dagli utenti.

Funzione:

Memorizza le librerie associate a ciascun utente, permettendo di organizzare i libri in collezioni.

Chiave primaria:

library_id (autoincrementale)

Dipendenze:

- Collegata a **Utente** tramite codiceFiscale
- Collegata a **Libro_libreria** tramite library_id

Libro_libreria

```
1 CREATE TABLE Libro_libreria(  
2 ll_id SERIAL PRIMARY key,  
3 book_id INT,  
4 library_id INT,  
5 FOREIGN KEY(book_id) REFERENCES Libro(book_id),  
6 FOREIGN KEY (library_id) REFERENCES Libreria(library_id)  
7 );
```

Descrizione:

Tabella di associazione tra libri e librerie.

Funzione:

Gestisce la relazione molti-a-molti tra **Libro** e **Libreria**, indicando quali libri sono presenti in ciascuna libreria.

Chiave primaria:

ll_id (autoincrementale)

Dipendenze:

- Collegata a **Libro** tramite book_id
- Collegata a **Libreria** tramite library_id

Valutazione

```
1 CREATE TABLE Valutazione (  
2     rating_id SERIAL PRIMARY KEY,  
3     stile INT,  
4     nota_stile VARCHAR(256),  
5     contenuto INT,  
6     nota_contenuto VARCHAR(256),  
7     gradevolezza INT,  
8     nota_gradevolezza VARCHAR(256),  
9     originalita INT,  
10    nota_originalita VARCHAR(256),  
11    edizione INT,  
12    nota_edizione VARCHAR(256),  
13    voto_finale INT,  
14    nota_voto_finale VARCHAR(256),  
15    book_id INT NOT NULL,  
16    codiceFiscale VARCHAR(16) NOT NULL,  
17    FOREIGN KEY (codiceFiscale) REFERENCES Utente(codiceFiscale),  
18    FOREIGN KEY (book_id) REFERENCES Libro(book_id)  
19 );
```

Descrizione:

Archivio delle valutazioni fornite dagli utenti sui libri.

Funzione:

Memorizza i giudizi dettagliati degli utenti sui libri, suddivisi per criteri (stile, contenuto, originalità, ecc.) e il voto finale.

Chiave primaria:

rating_id (autoincrementale)

Dipendenze:

- Collegata a **Utente** tramite codiceFiscale
- Collegata a **Libro** tramite book_id

LibriConsigliati

```
1 CREATE TABLE LibriConsigliati (  
2     recommended_id SERIAL PRIMARY KEY,  
3     CodiceFiscale VARCHAR(16) NOT NULL,  
4     book INT NOT NULL,  
5     book_recommended INT NOT NULL,  
6     FOREIGN KEY(codiceFiscale) REFERENCES Utente(codiceFiscale),  
7     FOREIGN KEY(book) REFERENCES Libro(book_id),  
8     FOREIGN KEY(book_recommended) REFERENCES Libro(book_id)  
9 );
```

Descrizione:

Elenco dei libri consigliati dagli utenti in relazione ad altri libri.

Funzione:

Memorizza le raccomandazioni effettuate dagli utenti, indicando un libro di riferimento e il libro consigliato.

Chiave primaria:

recommended_id (autoincrementale)

Dipendenze:

- Collegata a **Utente** tramite codiceFiscale
- Collegata a **Libro** tramite book
- Collegata a **Libro** tramite book_recommended

5. Architettura e implementazione

5.1 Scelte architetturali

Il sistema adotta diverse scelte architetturali:

- Architettura Client-Server Multi-thread:

```
public class Server{  
    private ServerSocket serverSocket;  
    public final static int PORT = 8080;  
    public Server(){  
        try{  
            serverSocket = new ServerSocket(PORT);  
        }catch(IOException e){}  
    }  
    public void esegui(){  
        while(true){  
            try{  
                System.out.println("Attendo una connessione");  
                Socket socket = serverSocket.accept();  
                System.out.println("Connessione effettuata");  
                new ServerMulti(socket);  
            }catch(IOException e){}  
        }  
    }  
}
```

- **Server centralizzato:** Un singolo server sulla porta 1090 gestisce tutte le richieste
- **Thread dedicati:** Ogni connessione client viene gestita da un ServerMulti thread separato
- **Comunicazione Socket TCP:** Utilizzo di socket per comunicazione affidabile
- **Separazione delle responsabilità:** Netta divisione tra logica di presentazione (client) e logica di business (server)

Proxy per la comunicazione

```
public class Proxy{
    private InetAddress ip;
    private Socket socket;
    private ObjectInputStream in;
    private ObjectOutputStream out;
    public Proxy(){
    }
    public synchronized void connect(){
        try{
            this.ip = InetAddress.getByName(null);
            socket = new Socket(ip,8080);
            this.out = new ObjectOutputStream(socket.getOutputStream());
            out.flush();
            this.in = new ObjectInputStream(socket.getInputStream());
            notifyAll();
        }catch(IOException e){
            e.printStackTrace();
        }
    }
    public synchronized void waitUntilConnected() {
        while (in == null || out == null) {
            try {
                wait();
            } catch (InterruptedException ignored) {}
        }
    }
}
```

Serializable

Tutte le classi parametro implementano l'interfaccia Serializable per permettere il trasferimento efficiente via socket. Questa scelta tecnica permette la trasmissione di oggetti complessi mantenendo l'integrità dei dati e le relazioni tra entità

esempio

```
public class Librerie implements Serializable{  
    private Utente utente;  
    private ArrayList<Libro> allLibri;  
    private String nome;
```


5.2 Gestione della comunicazione Client-Server

Protocollo di comunicazione basato su stringhe Il ServerMulti gestisce le richieste tramite un sistema di comandi string:

Esempio 1

```
public String getCodiceFiscale(String username,String password){
    String cf = "";
    try {
        out.writeObject("GetCodiceFiscale");
        out.flush();
        out.writeObject(username);
        out.flush();
        out.writeObject(password);
        out.flush();
        cf = (String)in.readObject();
    } catch (IOException | ClassNotFoundException e) {}
    return cf;
}
```

Esempio 2

```
public ValutazioniLibro getValutazioniLibro(Utente utente, Libro libro){
    try{
        out.writeObject("GetValutazioneLibro");
        out.flush();
    }
```

Esempio 3

```
public SuggerimentiLibro getSuggerimentiLibro(Utente utente, Libro libro){
    try{
        out.writeObject("getSuggerimentiLibro");
        out.flush();
    }
```

- **Estensibilità:** Facile aggiunta di nuove operazioni senza modificare l'architettura
- **Debug facilitato:** I comandi sono leggibili e tracciabili nei log
- **Flessibilità:** Permette operazioni con parametri variabili

Query SQL integrate nel ServerMulti

Tutte le query SQL sono integrate direttamente nel ServerMulti, permettendo ottimizzazioni specifiche per ogni operazione:

esempio query ricerca libri suggeriti

```
comandoQuery = "SELECT titolo, autore, anno_pubblicazione FROM libriConsigliati" +  
    "JOIN libro on (book_recommended = book_id)" +  
    "JOIN utente on (libriConsigliati.codicefiscale = utente.codicefiscale)" +  
    "WHERE libriconsigliati.codicefiscale = ? AND book_recommended = ?";
```

5.3 Operazioni Log In e Register

Register

La registrazione consente di inserire un nuovo utente nel sistema fornendo i dati personali richiesti.

Comportamento del sistema

- Se l'operazione va a buon fine → l'utente viene **registrato correttamente**
- In caso di errore (es. duplicati o problemi di connessione) → **la registrazione fallisce**

```
public boolean register(Utente u){  
    try {  
        out.writeObject("register");  
        out.flush();  
        out.writeObject(u);  
        out.flush();  
        return true;  
    } catch (IOException e) {  
        return false;  
    }  
}
```

Log In

La query di login verifica l'esistenza di un utente nel database confrontando username e password inseriti dall'utente.

Comportamento del sistema

- Se la query restituisce almeno una riga → **login valido**
- Se non restituisce risultati → **credenziali errate**

```
public boolean login(String username, String password){  
    try{  
        out.writeObject("login");  
        out.flush();  
        out.writeObject(username);  
        out.flush();  
        out.writeObject(password);  
        out.flush();  
        boolean b= (boolean)in.readObject();  
        return b;  
    }catch(Exception e){}  
    return false;  
}
```

5.4 Metodi di ricerca

Il sistema implementa tre modalità di ricerca per consentire agli utenti di trovare libri nel database in modo efficiente e flessibile.

Esempio 1

```
public ArrayList<Libro> ricercaPerTitolo(String titolo){  
    ArrayList<Libro> listaLibri = new ArrayList<>();  
    try {  
        out.writeObject("RicercaPerTitolo");  
        out.flush();  
        out.writeObject(titolo);  
        out.flush();  
    }
```

Esempio 2

```
public ArrayList<Libro> ricercaPerAutore(String autore){  
    ArrayList<Libro> listaLibri = new ArrayList<>();  
    try {  
        out.writeObject("RicercaPerAutore");  
        out.flush();  
        out.writeObject(autore);  
        out.flush();  
    }
```

Esempio 3

```
public ArrayList<Libro> ricercaPerAutoreEAnno(String autore, int anno){  
    ArrayList<Libro> listaLibri = new ArrayList<>();  
    try {  
        out.writeObject("CercaPerAutoreEAnno");  
        out.flush();  
        out.writeObject(autore);  
        out.flush();  
        out.writeObject(anno);  
        out.flush();  
    }
```

6. Limiti della soluzione sviluppata:

6.1 limiti di sicurezza

1. **Password Storage:** Password memorizzate in plain text nel database
2. **Comunicazione:** Nessuna crittografia per la comunicazione client-server

6.2 limiti funzionali

1. **Ricerca:** Funzionalità di ricerca limitate (solo per titolo/autore/autore e anno)
2. **Backup:** Nessun sistema automatico di backup database

6.3 limiti interfaccia grafica

1. **GUI:** Interfaccia non adattiva per diverse risoluzioni
2. **Lingue:** Nessun supporto per multiple lingue
3. **Mobile:** Nessun supporto per dispositivi mobili

6.4 limiti sistema operativo

1. **Dipendenza Windows:** Sviluppato e testato esclusivamente su Windows 11
2. **Path Windows:** Possibili problemi con separatori di path su sistemi Unix/Linux
3. **Font rendering:** Java Swing può avere comportamenti diversi su macOS e Linux

7. Sitografia:

7.1 tecnologie utilizzate

- Java SE: <https://docs.oracle.com/en/java/>
- Java Swing: <https://docs.oracle.com/javase/tutorial/uiswing/>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Supabase: <https://supabase.com/docs>
- JDBC: <https://docs.oracle.com/javase/tutorial/jdbc/>

7.2 programmazione socket e multi-thread

- Java Socket: <https://docs.oracle.com/javase/tutorial/networking/sockets/>
- Multi-threading: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>

7.3 DATABASE e UML

- PostgreSQL: <https://www.postgresqltutorial.com/>
- SQL Reference: <https://www.w3schools.com/sql/>
- Diagrams ER: <https://app.diagrams.net/>
- UML diagrams PlantUML: <https://editor.plantuml.com/>

7.4 GIT e GitHub

- Documentazione Git: <https://git-scm.com/doc>
- Documentazione GitHub: <https://docs.github.com/>

7.5 Sviluppo GUI

- Swing Layout: <https://docs.oracle.com/javase/tutorial/uiswing/layout/>
- Java AWT: <https://docs.oracle.com/javase/8/docs/api/java/awt/package-summary.html>
- Event Handling: <https://docs.oracle.com/javase/tutorial/uiswing/events/>
- Swing Components: <https://docs.oracle.com/javase/tutorial/uiswing/components/>